

PYTHON

Циклы

for · while · range · break · continue

```
for i in range(5):  
    print(i)
```

```
while x > 0:  
    x -= 1
```

Повторяем действие,
пока это имеет смысл.

Что сегодня научимся делать

02

Цель — выбирать подходящий цикл, а не просто запоминать синтаксис.

01 Повторять действие

for и while

02 Управлять циклом

break / continue

03 Работать с диапазонами

range(start, stop, step)

04 Решать задачи

счётчики, суммы, поиск

Зачем вообще нужны циклы?

Один и тот же алгоритм может повторяться тысячи раз.

Без цикла

```
print("Привет")  
print("Привет")  
print("Привет")  
print("Привет")
```

С циклом

```
for _ in range(4):  
    print("Привет")
```

Идея цикла: описать повторение один раз.

Цикл for

Используем, когда перебираем элементы или заранее понимаем число повторений.

```
for i in range(5):  
    print(i)
```

Результат

```
0  
1  
2  
3  
4
```

- `i` — переменная цикла
- `range(5)` создаёт значения 0, 1, 2, 3, 4
- тело цикла выполняется для каждого значения

Важно: правая граница `range` не включается.

range(): три формы

Диапазон можно настраивать.

range(5)

0, 1, 2, 3, 4

range(2, 7)

2, 3, 4, 5, 6

range(10, 2, -2)

10, 8, 6, 4

start — откуда · stop — до куда (не включая) · step — шаг

Практика 1 · предскажите вывод

Сначала думаем без запуска программы.

```
for i in range(2, 11, 3):  
    print(i * 2)
```

Что будет выведено?

Какие значения принимает i?

Ответ: 4, 10, 16

i: 2 → 5 → 8

Сумма и счётчик

Два базовых шаблона, которые встречаются постоянно.

```
total = 0
for i in range(1, 6):
    total += i
print(total)
```

Накопитель суммы

```
count = 0
for x in [4, 7, 2, 9]:
    if x > 5:
        count += 1
print(count)
```

Счётчик подходящих элементов

Перед циклом создаём переменную → внутри цикла обновляем.

Цикл while

Используем, когда повторяем действие, пока истинно условие.

```
x = 5
while x > 0:
    print(x)
    x -= 1
print('Старт!')
```

- проверяем условие
- если True — выполняем тело
- возвращаемся к условию
- если False — выходим

Главный риск: условие никогда не станет False → бесконечный цикл.

for или while?

Выбор определяется смыслом задачи.

for

- повторить N раз
- перебрать строку / список
- перебрать диапазон чисел

while

- повторять до события
- число повторений заранее неизвестно
- условие меняется во время работы

Если можно естественно сказать «для каждого...» → часто for. «Пока...» → while.

break и continue

Иногда обычного условия цикла недостаточно.

```
for x in range(1, 10):  
    if x == 5:  
        break  
    print(x)
```

break → полностью выйти из цикла

```
for x in range(1, 7):  
    if x % 2 == 0:  
        continue  
    print(x)
```

continue → перейти к следующей итерации

Вложенные циклы

Цикл может находиться внутри другого цикла.

```
for row in range(3):  
    for col in range(4):  
        print('*', end=' ')  
    print()
```

```
* * * *  
* * * *  
* * * *
```

Внешний цикл — строки. Внутренний — элементы строки.

3 × 4 = 12 выполнений внутреннего действия

Частые ошибки

12

Их полезно научиться видеть до запуска.

1

Забыли изменить переменную в while

`while x > 0: print(x)`

2

Ошиблись с границей range

`range(1, 10)` не включает 10

3

Сломали отступы

тело цикла определяется отступом

4

Изменяем не ту переменную

цикл не приближается к завершению